2D Shapes and Time

2

Graphics viewed on a computer screen are always made of pixels with different colours (images). Any shape to be displayed always undergoes a final stage where it is converted from coordinates of its vertices (and edges) to images. As previously said, the shapes are modelled by the relative positions of their vertices, that in turn are grouped for form polygons (triangles in most cases). A solid is then a set of polygons where the edges are not drawn but instead the inner region corresponding pixels are filled with colours, according to some colour/lighting model. The exercises below are related with modelling simple shapes and the application of some simple geometric transformations.

2.1 2D Shapes and Transformations

On the given triangle example notebook you can find an example that draw a triangle whose color varies from red to blue to green. In fact this is an automatic GL behaviour for interpolating information. Notice that the corner coordinates are in the range $[-1, 1]$, and as you can see on the second example when these coordinates are outside this range the result is a clipped image. Thus, you may retain this rule *only vertices with (final) coordinates x,y,z in the* $[-1, 1]$ *range are going to be drawn.* We will come to this subject later.

Exercises

For the following exercises you may just add cells copying the code and renaming the class name and canvas id, so that each exercise will be added bellow the previous ones.

Exercise 2.1

Readjust the vertices coordinates to form a vertical isosceles triangle, fully contained inside the canvas (or viewport).

Exercise 2.2
Add a red rectangle of side 0.8 and will be draw bellow the previous object. Use separate data structures for the vertices, and colours of the new object.

How can we make one object to be drawn below another?

Here we have two answers: 1) the last one to be drawn appears on top; 2) One object is made closer to the viewer than the other by adding a third Z coordinate to their vertices.

Show these possibilities and try the use of

— JavaScript —
```
this.gl.enable(this.gl.DEPTH_TEST);
```

and

— JavaScript —
```
this.gl.disable(this.gl.DEPTH_TEST);
```

2.2 Time

The introduction of time-dependent movements brings always a new dimension in the attractiveness of computer graphics. Frequently for the purpose of producing movement, programmers use counters that increase distances, angles, or other parameters. Despite that, the achieved results depend on how fast the computer is, or how heavy are the remaining computations. Thus, and depending on the computer speed, the increments may be done faster or slower and consequently the same happens to the afected movements (or evolutions).

To avoid this, it is important to use real time measures from the underlying operating system via some function available. Consider the use of the following pseudocode

— C++ —
```
oldTime = getTime();
...
newtime = getTime();
deltaT = newtime-oldtime;
oldtime = newtime;

newposition.x = SPEED*deltaT;
...
```

Now you can replace getTime() by some function provided by your preferred programming environment. For example in GLUT and SDL you can use respectively the number of milliseconds that passed since the initialisation of these toolkits

— C++ —

```
// in C/C++ + GLUT
int currentTime = glutGet(GLUT_ELAPSED_TIME);

// in C/C++ + SDL
int currentTime = SDL_GetTicks();

// in JavaScript
var currentTime = Date.now();
```

Exercises

Exercise 2.3

Using the current time information make the top vertice of the triangle move down till the middle of the window and up till the top at a rate of 0.5Hz.

Suggestion: Use the Math.sin(x) function.

Exercise 2.4

Make the triangle rotate by computing each vertex location by the application of

$$x' = x_{initial} \cos\theta(t) - y_{initial} \sin\theta(t)$$
$$y' = x_{initial} \sin\theta(t) + y_{initial} \cos\theta(t)$$

for each of them.

Exercise 2.5

Make the previous triangle rotate over a static rectangle/square as defined in exercise 2.2.

Exercise 2.6

Make the triangle also move up and down according to a sinus function of time by translating its vertices.

Exercise 2.7

What happens if you do the same but on the Z coordinates?

Note:

To solve this you must add a third coordinate to every vertex of the rectangles or triangles and add use the following:

Create a new shader

Create a new cell on the notebook and add the following code to it. Notice that it is basically equal to what is on one of the first cells of the triangles notebook with small differences. Can you spot and interpret the differences?

JavaScript

```
%%html
<script id="my-vertex-shader3D" type="x-shader/x-vertex">
precision highp float;

attribute  vec3 in_Position;
attribute  vec3 in_Color;

varying  vec3 ex_Color;

void main(void) {
    // Since we are using flat lines, our input only had two points: x and y.
    // Set the Z coordinate to 0 and W coordinate to 1

    gl_Position = vec4(in_Position.x, in_Position.y, in_Position.z , 1.0);

    // GLSL allows shorthand use of vectors too, the following is also valid:
    // gl_Position = vec4(in_Position, 0.0, 1.0);

    ex_Color = in_Color;
}
</script>
```

Initialization changes

Add the following code to the initialization of your class.

JavaScript

```
var fragsrc = document.getElementById("my-fragment-shader").text;
        var vertsrc = document.getElementById("my-vertex-shader3D").text;
        if(!fragsrc)
            console.log('Error: you forgot to define my-fragment-shader');
        if(!vertsrc)
            console.log('Error: you forgot to define my-vertex-shader');

        this.shaderprog = new DEECshader(this.gl);
        this.shaderprog.srcShaders(vertsrc,fragsrc);
```